

Détection d'Anomalies et Autoencodeurs

Application en Maintenance Prédictive

Cours Complet et Détaillé

Cours Approfondi
Machine Learning Non Supervisé

20 décembre 2025

Résumé

Ce cours présente de manière exhaustive les algorithmes de détection d'anomalies pour la maintenance prédictive industrielle. Nous couvrons en profondeur les **autoencodeurs** (Dense et LSTM) ainsi que les **méthodes classiques** (Isolation Forest, One-Class SVM, Local Outlier Factor).

Pour chaque méthode, nous présentons :

- Les fondements mathématiques rigoureux
- L'algorithme détaillé avec pseudocode
- Des exemples pratiques complets avec implémentation Python
- Des études de cas sur données réelles
- Les métriques d'évaluation appropriées
- Les bonnes pratiques et recommandations

Le cours inclut également des sections sur le clustering dans l'espace latent, l'évaluation des performances, et l'application pratique au dataset AI4I 2020 pour la maintenance prédictive.

Table des matières

1	Introduction à la Détection d'Anomalies	5
1.1	Contexte et Motivation	5
1.2	Définition Formelle	5
1.3	Types d'Anomalies	5
1.4	Approches de Détection	5
1.5	Défis et Enjeux	6
I	Autoencodeurs pour la Détection d'Anomalies	7
2	Fondements Théoriques des Autoencodeurs	7
2.1	Introduction et Motivation	7
2.2	Architecture Générale	7
2.3	Fonction de Perte	7
2.4	Régularisation	8
2.5	Détection d'Anomalies avec Autoencodeurs	8
3	Autoencodeur Dense (Feedforward)	8
3.1	Architecture Détaillée	8
3.1.1	Formulation Mathématique Complète	8
3.1.2	Fonctions d'Activation	9
3.2	Implémentation Complète avec TensorFlow/Keras	9
3.3	Analyse des Résultats	13
3.3.1	Métriques d'Évaluation	13
3.4	Avantages et Limitations	14
3.5	Bonnes Pratiques	15
4	Autoencodeur LSTM (pour séries temporelles)	16
II	Méthodes Classiques de Détection d'Anomalies	17
III	Clustering dans l'Espace Latent et Évaluation	18
5	Clustering Non Supervisé	18
6	Métriques d'Évaluation Avancées	18
IV	Application au Dataset AI4I 2020	19
7	Présentation du Dataset	19
8	Autoencodeur LSTM - Détails Complets	20
8.1	Architecture LSTM Détaillée	20
8.1.1	Problème du Gradient Vanishing	20
8.1.2	Solution LSTM	20
8.2	Implémentation LSTM Autoencoder	20
8.3	Applications des LSTM Autoencoders	21

V	Méthodes Classiques - Détails Complets	22
9	Isolation Forest - Analyse Approfondie	22
9.1	Théorie des Arbres d'Isolation	22
9.1.1	Propriétés Mathématiques	22
9.1.2	Score d'Anomalie Normalisé	22
9.2	Implémentation Complète	22
9.3	Analyse de Complexité	23
VI	Clustering et Analyse de l'Espace Latent	24
10	Clustering Non Supervisé	24
10.1	K-Means dans l'Espace Latent	24
10.1.1	Algorithme K-Means	24
10.1.2	Application à l'Espace Latent	24
10.2	DBSCAN pour Clustering Basé sur la Densité	25
VII	Étude de Cas : Dataset AI4I 2020	26
11	Présentation du Dataset	26
11.1	Description	26
11.2	Variables du Dataset	26
11.3	Analyse Exploratoire	26
12	Application des Algorithmes	27
12.1	Prétraitement	27
12.2	Comparaison des Méthodes	28
13	Exercices Pratiques	30
14	One-Class SVM - Analyse Mathématique Complète	31
14.1	Formulation du Problème d'Optimisation	31
14.1.1	Problème Primal	31
14.1.2	Problème Dual	31
14.2	Noyaux et Espaces de Hilbert	31
14.2.1	Théorie des Noyaux	31
14.3	Choix des Hyperparamètres	32
14.3.1	Paramètre ν	32
14.3.2	Paramètre γ (pour noyau RBF)	32
15	Local Outlier Factor - Analyse Complète	33
15.1	Fondements Théoriques	33
15.1.1	Motivation	33
15.2	Algorithme Détaillé	33
15.2.1	Étape 1 : k-distance	33
15.2.2	Étape 2 : k-voisinage	33
15.2.3	Étape 3 : Reachability Distance	33
15.2.4	Étape 4 : Local Reachability Density	33
15.2.5	Étape 5 : Local Outlier Factor	33
15.3	Interprétation du Score LOF	34
15.4	Complexité Algorithmique	34

16 Métriques d'Évaluation Avancées	35
16.1 Métriques pour Données Déséquilibrées	35
16.1.1 Precision-Recall Curve	35
16.1.2 F-beta Score	35
16.2 Métriques Spécifiques à la Détection d'Anomalies	35
16.2.1 Matthews Correlation Coefficient (MCC)	35
16.2.2 Cohen's Kappa	35
17 Visualisation et Interprétabilité	36
17.1 Réduction de Dimensionnalité	36
17.1.1 t-SNE (t-Distributed Stochastic Neighbor Embedding)	36
17.1.2 UMAP (Uniform Manifold Approximation and Projection)	36
17.2 SHAP Values pour l'Interprétabilité	37
18 Déploiement en Production	38
18.1 Pipeline MLOps	38
18.1.1 Architecture de Déploiement	38
18.1.2 Exemple d'API avec FastAPI	38
18.2 Monitoring et Drift Detection	39
18.2.1 Data Drift	39
19 Annexes	40
19.1 Ressources Complémentaires	40
19.1.1 Datasets Publics pour la Détection d'Anomalies	40
19.2 Bibliothèques Python Recommandées	40
19.3 Checklist de Projet	41
20 Études de Cas Supplémentaires	42
20.1 Cas 1 : Détection de Fraudes Bancaires	42
20.1.1 Contexte	42
20.1.2 Approche Recommandée	42
20.2 Cas 2 : Maintenance Prédictive d'Éoliennes	42
20.2.1 Contexte	42
20.2.2 Solution Proposée	42
21 Optimisation et Hyperparamètres	45
21.1 Grid Search et Random Search	45
21.1.1 Grid Search	45
21.1.2 Bayesian Optimization	45
21.2 AutoML pour Détection d'Anomalies	46
21.2.1 TPOT (Tree-based Pipeline Optimization Tool)	46
22 Détection d'Anomalies en Temps Réel	48
22.1 Architecture Streaming	48
22.1.1 Pipeline avec Apache Kafka	48
22.2 Détection Incrémentale	49
22.2.1 Online Learning	49
23 Conclusion et Perspectives	50
23.1 Récapitulatif	50
23.2 Tendances Futures	50
23.2.1 Deep Learning Avancé	50
23.2.2 Explainability (XAI)	50

23.2.3 AutoML et Neural Architecture Search	50
23.3 Recommandations Finales	51
23.4 Ressources pour Aller Plus Loin	51

1 Introduction à la Détection d'Anomalies

1.1 Contexte et Motivation

La détection d'anomalies est un problème fondamental en apprentissage automatique avec des applications critiques dans de nombreux domaines :

- **Maintenance prédictive** : Détecter les signes avant-coureurs de pannes d'équipements
- **Cybersécurité** : Identifier les intrusions et comportements malveillants
- **Finance** : Détecter les fraudes et transactions suspectes
- **Santé** : Identifier les pathologies rares ou nouvelles
- **Qualité industrielle** : Repérer les défauts de production

1.2 Définition Formelle

Définition 1.1 (Anomalie). *Une anomalie (ou outlier) est une observation qui s'écarte significativement du comportement normal ou attendu des données. Formellement, soit $\mathcal{X}$ l'espace des observations et $P(x)$ la distribution de probabilité des données normales. Un point $x \in \mathcal{X}$ est considéré comme une anomalie si :*

$$P(x) < \tau \quad (1)$$

où τ est un seuil de probabilité prédéfini.

1.3 Types d'Anomalies

1. **Anomalies ponctuelles (Point Anomalies)** : Une observation individuelle est anormale par rapport au reste des données.
2. **Anomalies contextuelles (Contextual Anomalies)** : Une observation est anormale dans un contexte spécifique mais normale dans d'autres contextes (ex : température de 30°C normale en été, anormale en hiver).
3. **Anomalies collectives (Collective Anomalies)** : Un ensemble d'observations est anormal, même si chaque observation individuelle peut sembler normale.

1.4 Approches de Détection

Approche	Principe	Exemples
Statistique	Modéliser la distribution normale	Gaussian Mixture, Z-score
Basée sur la distance	Mesurer l'éloignement	k-NN, LOF
Basée sur la densité	Estimer la densité locale	DBSCAN, LOF
Basée sur les modèles	Apprendre un modèle normal	Autoencodeurs, SVM
Basée sur les arbres	Isoler les points anormaux	Isolation Forest

TABLE 1 – Principales approches de détection d'anomalies

1.5 Défis et Enjeux

Attention

Défis majeurs en détection d'anomalies :

- **Déséquilibre des classes** : Les anomalies sont rares (souvent $< 1\%$ des données)
- **Absence d'étiquettes** : Approche non supervisée dans la plupart des cas
- **Évolution temporelle** : La notion de "normal" peut changer dans le temps
- **Haute dimensionnalité** : Malédiction de la dimensionnalité
- **Bruit vs Anomalies** : Distinguer le bruit des vraies anomalies

Première partie

Autoencodeurs pour la Détection d'Anomalies

2 Fondements Théoriques des Autoencodeurs

2.1 Introduction et Motivation

Les **autoencodeurs** sont des réseaux de neurones non supervisés conçus pour apprendre une représentation compacte (encodage) des données d'entrée, puis reconstruire ces données à partir de cette représentation.

Important

Principe fondamental : Un autoencodeur apprend une fonction d'identité $f(x) \approx x$ en passant par un goulot d'étranglement (bottleneck) qui force le réseau à capturer uniquement les caractéristiques essentielles des données.

2.2 Architecture Générale

Un autoencodeur se compose de trois parties principales :

1. **Encodeur** $f_{enc} : \mathcal{X} \rightarrow \mathcal{H}$

Transforme l'entrée $x \in \mathbb{R}^n$ vers une représentation latente $h \in \mathbb{R}^k$ avec $k < n$:

$$h = f_{enc}(x; \theta_{enc}) \quad (2)$$

2. **Espace latent** $\mathcal{H}$

Représentation compacte de dimension réduite qui capture l'essence des données

3. **Décodeur** $f_{dec} : \mathcal{H} \rightarrow \mathcal{X}$

Reconstruit l'entrée à partir de la représentation latente :

$$\hat{x} = f_{dec}(h; \theta_{dec}) \quad (3)$$

2.3 Fonction de Perte

L'objectif de l'entraînement est de minimiser l'erreur de reconstruction :

Définition 2.1 (Fonction de perte de reconstruction). *Pour un ensemble de données $\{x_1, x_2, \dots, x_N\}$, la fonction de perte est :*

$$\mathcal{L}(\theta_{enc}, \theta_{dec}) = \frac{1}{N} \sum_{i=1}^N \mathcal{D}(x_i, \hat{x}_i) \quad (4)$$

où $\mathcal{D}$ est une mesure de distance (MSE, cross-entropy, etc.).

Choix de la fonction de distance :

— **Mean Squared Error (MSE)** pour données continues :

$$\mathcal{D}_{MSE}(x, \hat{x}) = \|x - \hat{x}\|^2 = \sum_{j=1}^n (x_j - \hat{x}_j)^2 \quad (5)$$

— **Binary Cross-Entropy** pour données binaires :

$$\mathcal{D}_{BCE}(x, \hat{x}) = - \sum_{j=1}^n [x_j \log(\hat{x}_j) + (1 - x_j) \log(1 - \hat{x}_j)] \quad (6)$$

2.4 Régularisation

Pour éviter le surapprentissage et améliorer la généralisation :

$$\mathcal{L}_{total} = \mathcal{L}_{reconstruction} + \lambda \mathcal{L}_{regularization} \quad (7)$$

Types de régularisation :

1. **L2 (Weight Decay) :**

$$\mathcal{L}_{L2} = \sum_l ||W^{(l)}||_F^2 \quad (8)$$

2. **Sparse Autoencoder :** Encourage la parcimonie dans l'espace latent

$$\mathcal{L}_{sparse} = \beta \sum_{j=1}^k KL(\rho || \hat{\rho}_j) \quad (9)$$

où ρ est la sparsité cible et $\hat{\rho}_j$ l'activation moyenne du neurone j

3. **Denoising Autoencoder :** Ajoute du bruit à l'entrée pour forcer la robustesse

2.5 Détection d'Anomalies avec Autoencodeurs

Théorème 2.1 (Principe de détection). *Soit un autoencodeur f entraîné sur des données normales $\mathcal{X}_{normal}$. Pour une nouvelle observation x , l'erreur de reconstruction $e(x) = ||x - f(x)||^2$ est :*

- Faible si x est similaire aux données d'entraînement (normal)
- Élevée si x est différent des données d'entraînement (anomalie)

Seuil de détection :

$$\text{Anomalie}(x) = \begin{cases} 1 & \text{si } e(x) > \tau \\ 0 & \text{sinon} \end{cases} \quad (10)$$

Méthodes pour déterminer τ :

1. **Percentile** : $\tau = \text{percentile}_{95}(\{e(x_i) : x_i \in \mathcal{X}_{validation}\})$
2. **Écart-type** : $\tau = \mu_e + k\sigma_e$ avec $k \in [2, 3]$
3. **MAD (Median Absolute Deviation)** : Plus robuste aux outliers

3 Autoencodeur Dense (Feedforward)

3.1 Architecture Détaillée

Un autoencodeur dense utilise des couches fully-connected pour l'encodage et le décodage.

3.1.1 Formulation Mathématique Complète

Encodeur multi-couches :

$$h^{(1)} = \sigma_1(W^{(1)}x + b^{(1)}) \quad (11)$$

$$h^{(2)} = \sigma_2(W^{(2)}h^{(1)} + b^{(2)}) \quad (12)$$

$$\vdots \quad (13)$$

$$h^{(L_{enc})} = \sigma_{L_{enc}}(W^{(L_{enc})}h^{(L_{enc}-1)} + b^{(L_{enc})}) \quad (14)$$

Décodeur multi-couches :

$$\hat{h}^{(1)} = \sigma'_1(W'^{(1)}h^{(L_{enc})} + b'^{(1)}) \quad (15)$$

$$\hat{h}^{(2)} = \sigma'_2(W'^{(2)}\hat{h}^{(1)} + b'^{(2)}) \quad (16)$$

$$\vdots \quad (17)$$

$$\hat{x} = \sigma'_{L_{dec}}(W'^{(L_{dec})}\hat{h}^{(L_{dec}-1)} + b'^{(L_{dec})}) \quad (18)$$

3.1.2 Fonctions d'Activation

Activation	Formule	Usage
ReLU	$\sigma(x) = \max(0, x)$	Couches cachées
Leaky ReLU	$\sigma(x) = \max(\alpha x, x)$	Éviter dying ReLU
Tanh	$\sigma(x) = \tanh(x)$	Données normalisées [-1,1]
Sigmoid	$\sigma(x) = \frac{1}{1+e^{-x}}$	Sortie [0,1]
Linear	$\sigma(x) = x$	Sortie continue

TABLE 2 – Fonctions d'activation courantes

3.2 Implémentation Complète avec TensorFlow/Keras

```

1 import numpy as np
2 import tensorflow as tf
3 from tensorflow import keras
4 from tensorflow.keras import layers, regularizers
5 from sklearn.preprocessing import StandardScaler
6 from sklearn.model_selection import train_test_split
7 import matplotlib.pyplot as plt
8
9 # =====
10 # 1. PRÉPARATION DES DONNÉES
11 # =====
12
13 # Chargement des données (exemple avec AI4I 2020)
14 # Supposons que X contient les features normalisées
15 # et y contient les labels (0=normal, 1=faillure)
16
17 # Séparer données normales et anormales
18 X_normal = X[y == 0]
19 X_anomaly = X[y == 1]
20
21 # Split train/validation (uniquement données normales)
22 X_train, X_val = train_test_split(
23     X_normal, test_size=0.2, random_state=42
24 )
25
26 # Normalisation
27 scaler = StandardScaler()
28 X_train_scaled = scaler.fit_transform(X_train)
29 X_val_scaled = scaler.transform(X_val)
30 X_anomaly_scaled = scaler.transform(X_anomaly)
31
32 print(f"Données d'entraînement : {X_train_scaled.shape}")
33 print(f"Données de validation : {X_val_scaled.shape}")
34 print(f"Données anormales : {X_anomaly_scaled.shape}")
35

```

```

36 # =====
37 # 2. CONSTRUCTION DE L'AUTOENCODEUR
38 # =====
39
40 input_dim = X_train_scaled.shape[1]
41 encoding_dims = [64, 32, 16, 8] # Dimensions des couches
42
43 # Fonction pour cr er l'autoencodeur
44 def create_autoencoder(input_dim, encoding_dims, l2_reg=1e-4):
45     """
46     Cr e un autoencodeur dense avec architecture sym trique
47
48     Args:
49         input_dim: Dimension de l'entr e
50         encoding_dims: Liste des dimensions des couches d'encodage
51         l2_reg: Coefficient de r gularisation L2
52
53     Returns:
54         autoencoder: Mod le complet
55         encoder: Mod le encodeur seul
56     """
57     # ENCODEUR
58     encoder_input = layers.Input(shape=(input_dim,), name='input')
59     x = encoder_input
60
61     for i, dim in enumerate(encoding_dims):
62         x = layers.Dense(
63             dim,
64             activation='relu',
65             kernel_regularizer=regularizers.l2(l2_reg),
66             name=f'encoder_dense_{i+1}')
67         x = layers.BatchNormalization(name=f'encoder_bn_{i+1}')(x)
68         x = layers.Dropout(0.2, name=f'encoder_dropout_{i+1}')(x)
69
70     latent = x
71     encoder = keras.Model(encoder_input, latent, name='encoder')
72
73     # D CODEUR
74     decoder_input = layers.Input(shape=(encoding_dims[-1],), name='latent_input')
75     x = decoder_input
76
77     for i, dim in enumerate(reversed(encoding_dims[:-1])):
78         x = layers.Dense(
79             dim,
80             activation='relu',
81             kernel_regularizer=regularizers.l2(l2_reg),
82             name=f'decoder_dense_{i+1}')
83         x = layers.BatchNormalization(name=f'decoder_bn_{i+1}')(x)
84         x = layers.Dropout(0.2, name=f'decoder_dropout_{i+1}')(x)
85
86     # Couche de sortie
87     decoder_output = layers.Dense(
88         input_dim,
89         activation='linear',
90         name='output')
91     x = decoder_output
92
93     decoder = keras.Model(decoder_input, decoder_output, name='decoder')
94
95     # AUTOENCODEUR COMPLET

```

```

98     autoencoder_output = decoder(encoder(encoder_input))
99     autoencoder = keras.Model(
100         encoder_input,
101         autoencoder_output,
102         name='autoencoder'
103     )
104
105     return autoencoder, encoder
106
107 # Créer le modèle
108 autoencoder, encoder = create_autoencoder(input_dim, encoding_dims)
109
110 # Afficher l'architecture
111 autoencoder.summary()
112
113 # =====
114 # 3. COMPILATION ET CALLBACKS
115 # =====
116
117 # Compilation
118 autoencoder.compile(
119     optimizer=keras.optimizers.Adam(learning_rate=0.001),
120     loss='mse',
121     metrics=['mae']
122 )
123
124 # Callbacks
125 early_stopping = keras.callbacks.EarlyStopping(
126     monitor='val_loss',
127     patience=10,
128     restore_best_weights=True
129 )
130
131 reduce_lr = keras.callbacks.ReduceLROnPlateau(
132     monitor='val_loss',
133     factor=0.5,
134     patience=5,
135     min_lr=1e-6
136 )
137
138 # =====
139 # 4. ENTRAÎNEMENT
140 # =====
141
142 history = autoencoder.fit(
143     X_train_scaled, X_train_scaled,
144     epochs=100,
145     batch_size=32,
146     validation_data=(X_val_scaled, X_val_scaled),
147     callbacks=[early_stopping, reduce_lr],
148     verbose=1
149 )
150
151 # =====
152 # 5. VISUALISATION DE L'APPRENTISSAGE
153 # =====
154
155 fig, axes = plt.subplots(1, 2, figsize=(14, 5))
156
157 # Loss
158 axes[0].plot(history.history['loss'], label='Train Loss')
159 axes[0].plot(history.history['val_loss'], label='Val Loss')
160 axes[0].set_xlabel('Epoch')

```

```

161 axes[0].set_ylabel('MSE Loss')
162 axes[0].set_title(' Evolution de la perte')
163 axes[0].legend()
164 axes[0].grid(True, alpha=0.3)
165
166 # MAE
167 axes[1].plot(history.history['mae'], label='Train MAE')
168 axes[1].plot(history.history['val_mae'], label='Val MAE')
169 axes[1].set_xlabel('Epoch')
170 axes[1].set_ylabel('MAE')
171 axes[1].set_title(' Evolution du MAE')
172 axes[1].legend()
173 axes[1].grid(True, alpha=0.3)
174
175 plt.tight_layout()
176 plt.savefig('training_history.png', dpi=150)
177 plt.show()
178
179 # =====
180 # 6. DETECTION D'ANOMALIES
181 # =====
182
183 def compute_reconstruction_error(model, data):
184     """Calcule l'erreur de reconstruction"""
185     reconstructions = model.predict(data, verbose=0)
186     mse = np.mean(np.square(data - reconstructions), axis=1)
187     return mse
188
189 # Calculer les erreurs
190 errors_train = compute_reconstruction_error(autoencoder, X_train_scaled)
191 errors_val = compute_reconstruction_error(autoencoder, X_val_scaled)
192 errors_anomaly = compute_reconstruction_error(autoencoder, X_anomaly_scaled)
193
194 # Déterminer le seuil (95 me percentile)
195 threshold = np.percentile(errors_val, 95)
196
197 print(f"\n{'='*60}")
198 print(f"RÉSULTATS DE DETECTION")
199 print(f"{'='*60}")
200 print(f"Seuil (95 me percentile) : {threshold:.6f}")
201 print(f"Erreur moyenne (normal) : {errors_val.mean():.6f}")
202 print(f"Erreur moyenne (anomalies) : {errors_anomaly.mean():.6f}")
203
204 # Prédiction
205 pred_val = errors_val > threshold
206 pred_anomaly = errors_anomaly > threshold
207
208 print(f"\nFaux positifs : {pred_val.sum()}/{len(pred_val)} ({pred_val.mean()
209     *100:.1f}%)")
210
211 print(f"Vrais positifs : {pred_anomaly.sum()}/{len(pred_anomaly)} ({pred_anomaly
212     .mean()*100:.1f}%)")
213
214 # =====
215 # 7. VISUALISATIONS AVANCÉES
216 # =====
217
218 # Distribution des erreurs
219 fig, axes = plt.subplots(2, 2, figsize=(14, 10))
220
221 # Histogramme
222 axes[0, 0].hist(errors_val, bins=50, alpha=0.7, label='Normal', color='blue')
223 axes[0, 0].hist(errors_anomaly, bins=50, alpha=0.7, label='Anomalies', color='
red')

```

```

221 axes[0, 0].axvline(threshold, color='green', linestyle='--', linewidth=2, label=
    'Seuil')
222 axes[0, 0].set_xlabel('Erreur de reconstruction')
223 axes[0, 0].set_ylabel('Fréquence')
224 axes[0, 0].set_title('Distribution des erreurs')
225 axes[0, 0].legend()
226 axes[0, 0].grid(True, alpha=0.3)
227
228 # Box plot
229 axes[0, 1].boxplot([errors_val, errors_anomaly], labels=['Normal', 'Anomalies'])
230 axes[0, 1].axhline(threshold, color='green', linestyle='--', linewidth=2)
231 axes[0, 1].set_ylabel('Erreur de reconstruction')
232 axes[0, 1].set_title('Comparaison des erreurs')
233 axes[0, 1].grid(True, alpha=0.3)
234
235 # Espace latent (2 premières dimensions)
236 latent_val = encoder.predict(X_val_scaled, verbose=0)
237 latent_anomaly = encoder.predict(X_anomaly_scaled, verbose=0)
238
239 axes[1, 0].scatter(latent_val[:, 0], latent_val[:, 1],
240                   alpha=0.5, label='Normal', c='blue', s=20)
241 axes[1, 0].scatter(latent_anomaly[:, 0], latent_anomaly[:, 1],
242                   alpha=0.8, label='Anomalies', c='red', s=50, marker='x')
243 axes[1, 0].set_xlabel('Dimension latente 1')
244 axes[1, 0].set_ylabel('Dimension latente 2')
245 axes[1, 0].set_title('Espace latent (2D)')
246 axes[1, 0].legend()
247 axes[1, 0].grid(True, alpha=0.3)
248
249 # Courbe ROC
250 from sklearn.metrics import roc_curve, auc
251
252 y_true = np.concatenate([np.zeros(len(errors_val)), np.ones(len(errors_anomaly))
253 ])
254 y_scores = np.concatenate([errors_val, errors_anomaly])
255
256 fpr, tpr, thresholds = roc_curve(y_true, y_scores)
257 roc_auc = auc(fpr, tpr)
258
259 axes[1, 1].plot(fpr, tpr, color='darkorange', lw=2,
260                label=f'ROC curve (AUC = {roc_auc:.2f})')
261 axes[1, 1].plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
262 axes[1, 1].set_xlim([0.0, 1.0])
263 axes[1, 1].set_ylim([0.0, 1.05])
264 axes[1, 1].set_xlabel('Taux de faux positifs')
265 axes[1, 1].set_ylabel('Taux de vrais positifs')
266 axes[1, 1].set_title('Courbe ROC')
267 axes[1, 1].legend(loc="lower right")
268 axes[1, 1].grid(True, alpha=0.3)
269
270 plt.tight_layout()
271 plt.savefig('anomaly_detection_results.png', dpi=150)
272 plt.show()
273 print("\n Autoencodeur Dense : Analyse complétée terminée !")

```

Listing 1 – Autoencodeur Dense Complet avec Keras

3.3 Analyse des Résultats

3.3.1 Métriques d'Évaluation

Pour évaluer les performances de détection d'anomalies :

1. Matrice de confusion

		Prédiction	
		Normal	Anomalie
2*Vérité	Normal	TN	FP
	Anomalie	FN	TP

2. Précision (Precision)

$$\text{Precision} = \frac{TP}{TP + FP} \quad (19)$$

3. Rappel (Recall/Sensitivity)

$$\text{Recall} = \frac{TP}{TP + FN} \quad (20)$$

4. F1-Score

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (21)$$

5. AUC-ROC : Aire sous la courbe ROC

3.4 Avantages et Limitations

Astuce**Avantages de l'Autoencodeur Dense :**

- Capture les relations non-linéaires complexes
- Apprentissage non supervisé (pas besoin d'étiquettes)
- Espace latent utile pour visualisation et clustering
- Scalable avec GPU
- Flexible : architecture personnalisable

Attention**Limitations :**

- Nécessite beaucoup de données normales pour l'entraînement
- Sensible aux hyperparamètres (architecture, learning rate)
- Pas adapté aux séries temporelles (perd l'information séquentielle)
- Peut reconstruire certaines anomalies si elles sont simples
- Choix du seuil peut être délicat

3.5 Bonnes Pratiques

Astuce

Recommandations pour l'entraînement :

1. **Normalisation** : Toujours normaliser les données (StandardScaler, MinMaxScaler)
2. **Architecture** : Dimension latente = 10-30% de la dimension d'entrée
3. **Régularisation** : Utiliser L2, Dropout, Batch Normalization
4. **Early Stopping** : Éviter le surapprentissage
5. **Validation** : Utiliser un ensemble de validation pour choisir le seuil
6. **Ensemble** : Combiner plusieurs autoencodeurs pour plus de robustesse

4 Autoencodeur LSTM (pour séries temporelles)

[... contenu similaire mais plus détaillé pour LSTM ...]

Deuxième partie

Méthodes Classiques de Détection d'Anomalies

[... sections détaillées pour Isolation Forest, One-Class SVM, LOF ...]

Troisième partie

Clustering dans l'Espace Latent et Évaluation

5 Clustering Non Supervisé

[... section sur K-Means, DBSCAN dans l'espace latent ...]

6 Métriques d'Évaluation Avancées

[... section détaillée sur les métriques ...]

Quatrième partie

Application au Dataset AI4I 2020

7 Présentation du Dataset

[... étude de cas complète ...]

8 Autoencodeur LSTM - Détails Complets

8.1 Architecture LSTM Détaillée

Les LSTM (Long Short-Term Memory) sont des réseaux de neurones récurrents spécialement conçus pour traiter des séquences de données et capturer les dépendances à long terme.

8.1.1 Problème du Gradient Vanishing

Les RNN classiques souffrent du problème de gradient vanishing lors de la rétropropagation à travers le temps (BPTT). Pour une séquence de longueur T , le gradient est :

$$\frac{\partial \mathcal{L}}{\partial W} = \sum_{t=1}^T \frac{\partial \mathcal{L}_t}{\partial W} \quad (22)$$

Avec la règle de la chaîne, ce gradient peut devenir exponentiellement petit pour les dépendances lointaines.

8.1.2 Solution LSTM

Les LSTM résolvent ce problème avec une architecture à cellules mémoire et portes :

1. **Cellule mémoire** C_t : Stocke l'information à long terme
2. **État caché** h_t : Sortie à court terme
3. **Trois portes** : Contrôlent le flux d'information

Équations complètes :

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (\text{Forget gate}) \quad (23)$$

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (\text{Input gate}) \quad (24)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C) \quad (\text{Candidate}) \quad (25)$$

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \quad (\text{Cell state}) \quad (26)$$

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad (\text{Output gate}) \quad (27)$$

$$h_t = o_t \odot \tanh(C_t) \quad (\text{Hidden state}) \quad (28)$$

8.2 Implémentation LSTM Autoencoder

```

1 import numpy as np
2 import tensorflow as tf
3 from tensorflow import keras
4 from tensorflow.keras import layers
5
6 # Génération de séries temporelles synthétiques
7 def generate_timeseries(n_samples, n_timesteps, n_features):
8     data = []
9     for _ in range(n_samples):
10         series = np.zeros((n_timesteps, n_features))
11         for f in range(n_features):
12             freq = np.random.uniform(0.5, 2.0)
13             phase = np.random.uniform(0, 2*np.pi)
14             time = np.linspace(0, 10, n_timesteps)
15             series[:, f] = np.sin(2*np.pi*freq*time + phase)
16             series[:, f] += np.random.randn(n_timesteps) * 0.1
17         data.append(series)
18     return np.array(data)

```

```

19
20 # Param tres
21 n_timesteps = 100
22 n_features = 3
23 latent_dim = 10
24
25 # Donn es
26 X_train = generate_timeseries(500, n_timesteps, n_features)
27 X_val = generate_timeseries(100, n_timesteps, n_features)
28
29 # ENCODEUR LSTM
30 encoder_inputs = layers.Input(shape=(n_timesteps, n_features))
31 x = layers.LSTM(64, return_sequences=True)(encoder_inputs)
32 x = layers.LSTM(32, return_sequences=False)(x)
33 latent = layers.Dense(latent_dim, activation='relu')(x)
34
35 encoder = keras.Model(encoder_inputs, latent, name='encoder')
36
37 # D CODEUR LSTM
38 decoder_inputs = layers.Input(shape=(latent_dim,))
39 x = layers.RepeatVector(n_timesteps)(decoder_inputs)
40 x = layers.LSTM(32, return_sequences=True)(x)
41 x = layers.LSTM(64, return_sequences=True)(x)
42 decoder_outputs = layers.TimeDistributed(layers.Dense(n_features))(x)
43
44 decoder = keras.Model(decoder_inputs, decoder_outputs, name='decoder')
45
46 # AUTOENCODEUR COMPLET
47 autoencoder_inputs = layers.Input(shape=(n_timesteps, n_features))
48 encoded = encoder(autoencoder_inputs)
49 decoded = decoder(encoded)
50 lstm_ae = keras.Model(autoencoder_inputs, decoded, name='lstm_autoencoder')
51
52 # Compilation
53 lstm_ae.compile(optimizer='adam', loss='mse')
54
55 # Entra nement
56 history = lstm_ae.fit(
57     X_train, X_train,
58     epochs=50,
59     batch_size=32,
60     validation_data=(X_val, X_val),
61     verbose=1
62 )
63
64 print("LSTM Autoencoder entra n avec succ s!")

```

Listing 2 – LSTM Autoencoder Complet

8.3 Applications des LSTM Autoencoders

Exemple

Cas d'usage en maintenance prédictive :

- Détection de dérives dans les signaux de capteurs
- Prédiction de pannes basée sur les patterns temporels
- Identification de régimes de fonctionnement anormaux
- Analyse de vibrations pour la maintenance conditionnelle

Cinquième partie

Méthodes Classiques - Détails Complets

9 Isolation Forest - Analyse Approfondie

9.1 Théorie des Arbres d'Isolation

9.1.1 Propriétés Mathématiques

Proposition 9.1 (Longueur de chemin attendue). *Pour un arbre binaire de recherche aléatoire avec n nœuds, la longueur de chemin moyenne est :*

$$c(n) = 2H(n-1) - \frac{2(n-1)}{n} \quad (29)$$

où $H(i)$ est le i -ème nombre harmonique : $H(i) = \ln(i) + 0.5772$ (constante d'Euler).

9.1.2 Score d'Anomalie Normalisé

Le score d'anomalie $s(x, n)$ est normalisé entre 0 et 1 :

$$s(x, n) = 2^{-\frac{E(h(x))}{c(n)}} \quad (30)$$

Interprétation :

- Si $E(h(x)) \rightarrow 0 : s \rightarrow 1$ (anomalie certaine)
- Si $E(h(x)) \rightarrow c(n) : s \rightarrow 0.5$ (comportement normal)
- Si $E(h(x)) \rightarrow n-1 : s \rightarrow 0$ (très normal)

9.2 Implémentation Complète

```

1 from sklearn.ensemble import IsolationForest
2 from sklearn.datasets import make_blobs
3 from sklearn.metrics import classification_report, roc_auc_score
4 import numpy as np
5 import matplotlib.pyplot as plt
6
7 # Génération de données
8 X_normal, _ = make_blobs(n_samples=300, centers=2, n_features=2,
9                          cluster_std=0.5, random_state=42)
10 X_anomaly = np.random.uniform(low=-8, high=8, size=(20, 2))
11
12 X = np.vstack([X_normal, X_anomaly])
13 y_true = np.array([0]*len(X_normal) + [1]*len(X_anomaly))
14
15 # Modèle
16 iso_forest = IsolationForest(
17     n_estimators=100,
18     max_samples=256,
19     contamination=0.1,
20     random_state=42
21 )
22
23 # Entraînement et prédiction
24 y_pred = iso_forest.fit_predict(X)
25 scores = iso_forest.score_samples(X)
26
27 # Conversion des prédictions

```

```

28 y_pred_binary = (y_pred == -1).astype(int)
29
30 # valuation
31 print("Classification Report:")
32 print(classification_report(y_true, y_pred_binary))
33
34 # AUC-ROC
35 auc = roc_auc_score(y_true, -scores)
36 print(f"\nAUC-ROC: {auc:.4f}")
37
38 # Visualisation
39 fig, axes = plt.subplots(1, 2, figsize=(14, 6))
40
41 # Plot 1: Données avec vérité terrain
42 axes[0].scatter(X[y_true==0, 0], X[y_true==0, 1],
43                c='blue', label='Normal', alpha=0.6)
44 axes[0].scatter(X[y_true==1, 0], X[y_true==1, 1],
45                c='red', label='Anomalie', marker='x', s=100)
46 axes[0].set_title('Vérité Terrain')
47 axes[0].legend()
48 axes[0].grid(True, alpha=0.3)
49
50 # Plot 2: Prédiction avec scores
51 scatter = axes[1].scatter(X[:, 0], X[:, 1], c=-scores,
52                          cmap='RdYlBu', s=50, alpha=0.7)
53 axes[1].scatter(X[y_pred_binary==1, 0], X[y_pred_binary==1, 1],
54                edgecolors='red', facecolors='none', s=150, linewidths=2)
55 axes[1].set_title('Prédiction Isolation Forest')
56 plt.colorbar(scatter, ax=axes[1], label='Score d\'anomalie')
57 axes[1].grid(True, alpha=0.3)
58
59 plt.tight_layout()
60 plt.savefig('isolation_forest_detailed.png', dpi=150)
61 plt.show()

```

Listing 3 – Isolation Forest avec Analyse Détaillée

9.3 Analyse de Complexité

Opération	Complexité	Explication
Construction d'un arbre	$O(n \log n)$	Divisions récursives
Ensemble de t arbres	$O(t \cdot n \log n)$	t arbres indépendants
Prédiction pour un point	$O(t \cdot \log n)$	Parcours de t arbres
Prédiction pour m points	$O(t \cdot m \cdot \log n)$	m prédictions

TABLE 3 – Complexité algorithmique d'Isolation Forest

Sixième partie

Clustering et Analyse de l'Espace Latent

10 Clustering Non Supervisé

10.1 K-Means dans l'Espace Latent

10.1.1 Algorithme K-Means

Algorithm 1 K-Means Clustering

```

1: Entrée : Données  $X = \{x_1, \dots, x_n\}$ , nombre de clusters  $k$ 
2: Sortie : Centres  $\mu_1, \dots, \mu_k$  et assignations  $c_1, \dots, c_n$ 
3: Initialiser aléatoirement les centres  $\mu_1, \dots, \mu_k$ 
4: repeat
5:   Étape E (Expectation) :
6:   for  $i = 1$  to  $n$  do
7:      $c_i \leftarrow \arg \min_j \|x_i - \mu_j\|^2$ 
8:   end for
9:   Étape M (Maximization) :
10:  for  $j = 1$  to  $k$  do
11:     $\mu_j \leftarrow \frac{1}{|C_j|} \sum_{i \in C_j} x_i$ 
12:  end for
13: until convergence

```

10.1.2 Application à l'Espace Latent

```

1 from sklearn.cluster import KMeans, DBSCAN
2 from sklearn.metrics import silhouette_score
3 import matplotlib.pyplot as plt
4
5 # Supposons que nous avons un encodeur entra n
6 latent_representations = encoder.predict(X_normal_scaled)
7
8 # K-Means
9 n_clusters = 3
10 kmeans = KMeans(n_clusters=n_clusters, random_state=42)
11 cluster_labels = kmeans.fit_predict(latent_representations)
12
13 # Score de silhouette
14 silhouette_avg = silhouette_score(latent_representations, cluster_labels)
15 print(f"Score de silhouette: {silhouette_avg:.4f}")
16
17 # Visualisation (2D projection)
18 from sklearn.decomposition import PCA
19
20 pca = PCA(n_components=2)
21 latent_2d = pca.fit_transform(latent_representations)
22
23 plt.figure(figsize=(10, 6))
24 scatter = plt.scatter(latent_2d[:, 0], latent_2d[:, 1],
25                       c=cluster_labels, cmap='viridis', s=50, alpha=0.6)
26 plt.scatter(kmeans.cluster_centers_[0], kmeans.cluster_centers_[1],
27           c='red', marker='X', s=200, edgecolors='black', linewidths=2,
28           label='Centres')
29 plt.xlabel('Composante principale 1')

```

```
30 plt.ylabel('Composante principale 2')
31 plt.title(f'Clustering K-Means (k={n_clusters}, Silhouette={silhouette_avg:.3f})')
32 plt.colorbar(scatter, label='Cluster')
33 plt.legend()
34 plt.grid(True, alpha=0.3)
35 plt.savefig('clustering_latent_space.png', dpi=150)
36 plt.show()
```

Listing 4 – Clustering dans l'Espace Latent

10.2 DBSCAN pour Clustering Basé sur la Densité

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) identifie des clusters de forme arbitraire.

Paramètres :

- ϵ : Rayon de voisinage
- *minPts* : Nombre minimum de points pour former un cluster dense

Types de points :

1. **Core point** : Au moins *minPts* voisins dans un rayon ϵ
2. **Border point** : Moins de *minPts* voisins, mais dans le voisinage d'un core point
3. **Noise point** : Ni core ni border (outlier)

Septième partie

Étude de Cas : Dataset AI4I 2020

11 Présentation du Dataset

11.1 Description

Le dataset AI4I 2020 Predictive Maintenance contient des données synthétiques de maintenance prédictive reflétant des scénarios industriels réels.

Caractéristiques :

- **Taille** : 10,000 observations
- **Features** : 14 variables (6 numériques, 8 catégorielles)
- **Target** : Failure (binaire : 0=normal, 1=panne)
- **Types de pannes** : 5 modes de défaillance

11.2 Variables du Dataset

Variable	Type	Description
UID	Identifiant	Identifiant unique
Product ID	Catégorielle	Type de produit (L/M/H)
Type	Catégorielle	Qualité du produit
Air temperature [K]	Numérique	Température de l'air (K)
Process temperature [K]	Numérique	Température du processus (K)
Rotational speed [rpm]	Numérique	Vitesse de rotation (rpm)
Torque [Nm]	Numérique	Couple (Nm)
Tool wear [min]	Numérique	Usure de l'outil (min)
Machine failure	Binaire	Panne (0/1)
TWF	Binaire	Tool Wear Failure
HDF	Binaire	Heat Dissipation Failure
PWF	Binaire	Power Failure
OSF	Binaire	Overstrain Failure
RNF	Binaire	Random Failures

TABLE 4 – Variables du dataset AI4I 2020

11.3 Analyse Exploratoire

```

1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5
6 # Chargement
7 df = pd.read_csv('ai4i2020.csv')
8
9 print("Shape:", df.shape)
10 print("\nInfo:")
11 print(df.info())
12
13 print("\nStatistiques descriptives:")
14 print(df.describe())
15
16 # Distribution de la target
17 print("\nDistribution des pannes:")

```

```

18 print(df['Machine failure'].value_counts())
19 print(f"Taux de pannes: {df['Machine failure'].mean()*100:.2f}%")
20
21 # Corr lations
22 numeric_cols = df.select_dtypes(include=[np.number]).columns
23 correlation_matrix = df[numeric_cols].corr()
24
25 plt.figure(figsize=(12, 10))
26 sns.heatmap(correlation_matrix, annot=True, fmt='.2f',
27             cmap='coolwarm', center=0, square=True)
28 plt.title('Matrice de Corr lation')
29 plt.tight_layout()
30 plt.savefig('correlation_matrix.png', dpi=150)
31 plt.show()
32
33 # Distribution des features
34 fig, axes = plt.subplots(2, 3, figsize=(15, 10))
35 axes = axes.ravel()
36
37 numeric_features = ['Air temperature [K]', 'Process temperature [K]',
38                    'Rotational speed [rpm]', 'Torque [Nm]', 'Tool wear [min]']
39
40 for i, col in enumerate(numeric_features):
41     axes[i].hist(df[col], bins=50, alpha=0.7, edgecolor='black')
42     axes[i].set_title(f'Distribution de {col}')
43     axes[i].set_xlabel(col)
44     axes[i].set_ylabel('Fr quence')
45     axes[i].grid(True, alpha=0.3)
46
47 plt.tight_layout()
48 plt.savefig('feature_distributions.png', dpi=150)
49 plt.show()

```

Listing 5 – Chargement et Exploration du Dataset

12 Application des Algorithmes

12.1 Prétraitement

```

1 from sklearn.preprocessing import StandardScaler, LabelEncoder
2 from sklearn.model_selection import train_test_split
3
4 # S lection des features
5 feature_cols = ['Air temperature [K]', 'Process temperature [K]',
6                'Rotational speed [rpm]', 'Torque [Nm]', 'Tool wear [min]']
7
8 X = df[feature_cols].values
9 y = df['Machine failure'].values
10
11 # S paration donn es normales/anormales
12 X_normal = X[y == 0]
13 X_failure = X[y == 1]
14
15 print(f"Donn es normales: {len(X_normal)}")
16 print(f"Donn es avec pannes: {len(X_failure)}")
17
18 # Split train/val (uniquement donn es normales)
19 X_train, X_val = train_test_split(X_normal, test_size=0.2, random_state=42)
20
21 # Normalisation
22 scaler = StandardScaler()

```

```

23 X_train_scaled = scaler.fit_transform(X_train)
24 X_val_scaled = scaler.transform(X_val)
25 X_failure_scaled = scaler.transform(X_failure)
26
27 print("\nDonn es pr trait es:")
28 print(f"Train: {X_train_scaled.shape}")
29 print(f"Validation: {X_val_scaled.shape}")
30 print(f"Failures: {X_failure_scaled.shape}")

```

Listing 6 – Prétraitement du Dataset AI4I 2020

12.2 Comparaison des Méthodes

```

1 from sklearn.ensemble import IsolationForest
2 from sklearn.svm import OneClassSVM
3 from sklearn.neighbors import LocalOutlierFactor
4 from sklearn.metrics import precision_score, recall_score, f1_score,
   roc_auc_score
5
6 # Dictionnaire pour stocker les r sultats
7 results = {}
8
9 # 1. ISOLATION FOREST
10 iso_forest = IsolationForest(n_estimators=100, contamination=0.1, random_state
   =42)
11 iso_pred = iso_forest.fit_predict(np.vstack([X_val_scaled, X_failure_scaled]))
12 iso_scores = iso_forest.score_samples(np.vstack([X_val_scaled, X_failure_scaled
   ]))
13
14 # 2. ONE-CLASS SVM
15 ocsvm = OneClassSVM(kernel='rbf', gamma='auto', nu=0.1)
16 ocsvm.fit(X_train_scaled)
17 ocsvm_pred = ocsvm.predict(np.vstack([X_val_scaled, X_failure_scaled]))
18 ocsvm_scores = ocsvm.decision_function(np.vstack([X_val_scaled, X_failure_scaled
   ]))
19
20 # 3. LOCAL OUTLIER FACTOR
21 lof = LocalOutlierFactor(n_neighbors=20, contamination=0.1, novelty=True)
22 lof.fit(X_train_scaled)
23 lof_pred = lof.predict(np.vstack([X_val_scaled, X_failure_scaled]))
24 lof_scores = lof.decision_function(np.vstack([X_val_scaled, X_failure_scaled]))
25
26 # V rit terrain
27 y_true = np.concatenate([np.zeros(len(X_val_scaled)), np.ones(len(
   X_failure_scaled))])
28
29 # Conversion des pr dictions
30 methods = {
31     'Isolation Forest': (iso_pred == -1).astype(int),
32     'One-Class SVM': (ocsvm_pred == -1).astype(int),
33     'LOF': (lof_pred == -1).astype(int)
34 }
35
36 scores_dict = {
37     'Isolation Forest': -iso_scores,
38     'One-Class SVM': -ocsvm_scores,
39     'LOF': -lof_scores
40 }
41
42 # valuation
43 print("\n" + "="*70)
44 print("COMPARAISON DES M THODES SUR AI4I 2020")

```

```

45 print("="*70)
46 print(f"{'M thode':<20} {'Pr cision':<12} {'Rappel':<12} {'F1-Score':<12} {'AUC-ROC':<12}")
47 print("-"*70)
48
49 for name, y_pred in methods.items():
50     precision = precision_score(y_true, y_pred)
51     recall = recall_score(y_true, y_pred)
52     f1 = f1_score(y_true, y_pred)
53     auc = roc_auc_score(y_true, scores_dict[name])
54
55     print(f"{name:<20} {precision:<12.4f} {recall:<12.4f} {f1:<12.4f} {auc:<12.4f}")
56
57     results[name] = {
58         'precision': precision,
59         'recall': recall,
60         'f1': f1,
61         'auc': auc
62     }
63
64 # Visualisation comparative
65 fig, axes = plt.subplots(1, 3, figsize=(18, 5))
66
67 for idx, (name, scores) in enumerate(scores_dict.items()):
68     from sklearn.metrics import roc_curve
69
70     fpr, tpr, _ = roc_curve(y_true, scores)
71     auc_score = roc_auc_score(y_true, scores)
72
73     axes[idx].plot(fpr, tpr, linewidth=2, label=f'AUC = {auc_score:.4f}')
74     axes[idx].plot([0, 1], [0, 1], 'k--', linewidth=1)
75     axes[idx].set_xlabel('Taux de Faux Positifs')
76     axes[idx].set_ylabel('Taux de Vrais Positifs')
77     axes[idx].set_title(f'Courbe ROC - {name}')
78     axes[idx].legend()
79     axes[idx].grid(True, alpha=0.3)
80
81 plt.tight_layout()
82 plt.savefig('roc_comparison_ai4i.png', dpi=150)
83 plt.show()

```

Listing 7 – Comparaison Complète des Algorithmes

13 Exercices Pratiques

Exercice

Exercice 1 : Implémentation d'un Autoencodeur Simple

Créez un autoencodeur dense pour détecter les anomalies dans le dataset AI4I 2020.

Consignes :

1. Chargez le dataset et sélectionnez les features numériques
2. Créez un autoencodeur avec architecture [5, 3, 2, 3, 5]
3. Entraînez sur les données normales uniquement
4. Calculez l'erreur de reconstruction
5. Déterminez un seuil optimal
6. Évaluez les performances (Precision, Recall, F1, AUC)

Exercice

Exercice 2 : Comparaison des Méthodes

Comparez Isolation Forest, One-Class SVM et LOF sur le même dataset.

Questions :

1. Quelle méthode donne le meilleur AUC-ROC ?
2. Quelle méthode est la plus rapide à entraîner ?
3. Comment varie la performance avec le paramètre de contamination ?
4. Visualisez les frontières de décision (si possible en 2D)

Exercice

Exercice 3 : Clustering dans l'Espace Latent

Utilisez un autoencodeur pour créer un espace latent, puis appliquez K-Means.

Objectifs :

1. Entraînez un autoencodeur et extrayez l'espace latent
2. Appliquez K-Means avec $k=3$
3. Calculez le score de silhouette
4. Visualisez les clusters avec PCA ou t-SNE
5. Interprétez les clusters (régimes de fonctionnement ?)

14 One-Class SVM - Analyse Mathématique Complète

14.1 Formulation du Problème d'Optimisation

14.1.1 Problème Primal

Le One-Class SVM cherche à trouver un hyperplan qui sépare les données de l'origine dans un espace transformé par un noyau, tout en maximisant la marge.

Problème d'optimisation primal :

$$\min_{w, \rho, \xi} \frac{1}{2} \|w\|^2 - \rho + \frac{1}{\nu n} \sum_{i=1}^n \xi_i \quad (31)$$

sous contraintes :

$$w^T \phi(x_i) \geq \rho - \xi_i, \quad \forall i = 1, \dots, n \quad (32)$$

$$\xi_i \geq 0, \quad \forall i = 1, \dots, n \quad (33)$$

où :

- $w \in \mathcal{H}$: vecteur normal à l'hyperplan dans l'espace des features
- $\rho \in \mathbb{R}$: offset (distance à l'origine)
- $\xi_i \geq 0$: variables de relâchement (slack variables)
- $\nu \in (0, 1)$: paramètre contrôlant le compromis entre marge et violations
- $\phi : \mathcal{X} \rightarrow \mathcal{H}$: transformation via noyau

14.1.2 Problème Dual

En utilisant les multiplicateurs de Lagrange, on obtient le problème dual :

$$\max_{\alpha} -\frac{1}{2} \sum_{i,j=1}^n \alpha_i \alpha_j K(x_i, x_j) \quad (34)$$

sous contraintes :

$$0 \leq \alpha_i \leq \frac{1}{\nu n}, \quad \forall i = 1, \dots, n \quad (35)$$

$$\sum_{i=1}^n \alpha_i = 1 \quad (36)$$

Fonction de décision :

$$f(x) = \text{sign} \left(\sum_{i=1}^n \alpha_i K(x_i, x) - \rho \right) \quad (37)$$

14.2 Noyaux et Espaces de Hilbert

14.2.1 Théorie des Noyaux

Définition 14.1 (Noyau défini positif). *Une fonction $K : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$ est un noyau défini positif s'il existe un espace de Hilbert $\mathcal{H}$ et une application $\phi : \mathcal{X} \rightarrow \mathcal{H}$ tels que :*

$$K(x, x') = \langle \phi(x), \phi(x') \rangle_{\mathcal{H}} \quad (38)$$

Noyaux courants :

1. **Noyau Linéaire :**

$$K(x, x') = \langle x, x' \rangle \quad (39)$$

2. **Noyau Polynomial :**

$$K(x, x') = (\gamma \langle x, x' \rangle + r)^d \quad (40)$$

où $\gamma > 0$, $r \geq 0$, $d \in \mathbb{N}$

3. **Noyau RBF (Gaussian) :**

$$K(x, x') = \exp(-\gamma \|x - x'\|^2) \quad (41)$$

où $\gamma = \frac{1}{2\sigma^2}$

4. **Noyau Sigmoidal :**

$$K(x, x') = \tanh(\gamma \langle x, x' \rangle + r) \quad (42)$$

14.3 Choix des Hyperparamètres

14.3.1 Paramètre ν

Le paramètre ν a une interprétation probabiliste :

Théorème 14.1 (Propriétés de ν). *Le paramètre $\nu \in (0, 1)$ satisfait :*

1. ν est une borne supérieure sur la fraction de points en dehors de la frontière
2. ν est une borne inférieure sur la fraction de vecteurs support

Recommandations pratiques :

- $\nu = 0.01$: Très peu d'outliers attendus (1%)
- $\nu = 0.05$: Peu d'outliers (5%)
- $\nu = 0.1$: Modéré (10%)

14.3.2 Paramètre γ (pour noyau RBF)

Le paramètre γ contrôle la "portée" du noyau :

- γ **petit** : Influence à longue portée, frontière lisse
- γ **grand** : Influence locale, frontière complexe

Heuristique : $\gamma = \frac{1}{n_{features} \times \text{var}(X)}$

15 Local Outlier Factor - Analyse Complète

15.1 Fondements Théoriques

15.1.1 Motivation

LOF détecte les anomalies **locales** en comparant la densité locale d'un point avec celle de ses voisins. Contrairement aux méthodes globales, LOF peut identifier des points anormaux dans des régions de densité variable.

15.2 Algorithme Détaillé

15.2.1 Étape 1 : k-distance

Définition 15.1 (k-distance). *Pour un point p et un entier k , la k -distance de p est la distance au k -ième plus proche voisin :*

$$k\text{-distance}(p) = d(p, o_k) \quad (43)$$

où o_k est le k -ième plus proche voisin de p .

15.2.2 Étape 2 : k-voisinage

Définition 15.2 (k-voisinage). *Le k -voisinage de p est l'ensemble des points à distance au plus $k\text{-distance}(p)$:*

$$N_k(p) = \{q \in D \setminus \{p\} : d(p, q) \leq k\text{-distance}(p)\} \quad (44)$$

Note : $|N_k(p)| \geq k$ (peut être strictement supérieur en cas d'ex-aequo).

15.2.3 Étape 3 : Reachability Distance

Définition 15.3 (Reachability distance). *La reachability distance entre p et o est :*

$$\text{reach-dist}_k(p, o) = \max\{k\text{-distance}(o), d(p, o)\} \quad (45)$$

Intuition : Cette distance "lisse" les fluctuations statistiques pour les points très proches.

15.2.4 Étape 4 : Local Reachability Density

Définition 15.4 (LRD). *La densité locale de reachability de p est :*

$$LRD_k(p) = \frac{1}{\frac{\sum_{o \in N_k(p)} \text{reach-dist}_k(p, o)}{|N_k(p)|}} \quad (46)$$

C'est l'inverse de la distance moyenne de reachability aux voisins.

15.2.5 Étape 5 : Local Outlier Factor

Définition 15.5 (LOF). *Le Local Outlier Factor de p est :*

$$LOF_k(p) = \frac{\sum_{o \in N_k(p)} \frac{LRD_k(o)}{LRD_k(p)}}{|N_k(p)|} \quad (47)$$

15.3 Interprétation du Score LOF

Valeur LOF	Interprétation
$\text{LOF} \approx 1$	Densité similaire aux voisins (normal)
$\text{LOF} > 1$	Densité plus faible que les voisins
$\text{LOF} \gg 1$	Anomalie locale forte
$\text{LOF} < 1$	Densité plus élevée que les voisins

TABLE 5 – Interprétation des valeurs LOF

15.4 Complexité Algorithmique

Opération	Complexité
Calcul des k-distances	$O(n^2)$ (naïf) ou $O(n \log n)$ (avec index)
Calcul des LRD	$O(nk)$
Calcul des LOF	$O(nk)$
Total	$O(n^2)$ ou $O(n \log n + nk)$

TABLE 6 – Complexité de LOF

16 Métriques d'Évaluation Avancées

16.1 Métriques pour Données Déséquilibrées

16.1.1 Precision-Recall Curve

Pour les problèmes avec fort déséquilibre de classes, la courbe Precision-Recall est plus informative que la courbe ROC.

Average Precision (AP) :

$$AP = \sum_n (R_n - R_{n-1}) P_n \quad (48)$$

où P_n et R_n sont la précision et le rappel au n -ième seuil.

16.1.2 F-beta Score

Généralisation du F1-score avec pondération du rappel :

$$F_\beta = (1 + \beta^2) \times \frac{\text{Precision} \times \text{Recall}}{\beta^2 \times \text{Precision} + \text{Recall}} \quad (49)$$

- $\beta < 1$: Favorise la précision
- $\beta = 1$: F1-score (équilibre)
- $\beta > 1$: Favorise le rappel

En maintenance prédictive : Souvent $\beta = 2$ (privilégier le rappel pour ne pas manquer de pannes).

16.2 Métriques Spécifiques à la Détection d'Anomalies

16.2.1 Matthews Correlation Coefficient (MCC)

Métrique robuste pour données déséquilibrées :

$$MCC = \frac{TP \times TN - FP \times FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}} \quad (50)$$

Valeurs : $MCC \in [-1, 1]$

- $MCC = 1$: Prédiction parfaite
- $MCC = 0$: Prédiction aléatoire
- $MCC = -1$: Désaccord total

16.2.2 Cohen's Kappa

Mesure l'accord entre prédictions et vérité terrain, corrigé du hasard :

$$\kappa = \frac{p_o - p_e}{1 - p_e} \quad (51)$$

où :

- p_o : Accord observé
- p_e : Accord attendu par hasard

17 Visualisation et Interprétabilité

17.1 Réduction de Dimensionnalité

17.1.1 t-SNE (t-Distributed Stochastic Neighbor Embedding)

t-SNE préserve la structure locale des données en haute dimension lors de la projection en 2D/3D.

Algorithme :

1. Calculer les similarités en haute dimension (distribution gaussienne)
2. Initialiser les positions en basse dimension
3. Optimiser pour minimiser la divergence KL entre les deux distributions

Fonction objectif :

$$C = \sum_i KL(P_i || Q_i) = \sum_i \sum_j p_{ij} \log \frac{p_{ij}}{q_{ij}} \quad (52)$$

```

1 from sklearn.manifold import TSNE
2 import matplotlib.pyplot as plt
3
4 # Supposons que nous avons l'espace latent
5 latent_representations = encoder.predict(X_normal_scaled)
6
7 # t-SNE
8 tsne = TSNE(n_components=2, perplexity=30, random_state=42)
9 latent_2d = tsne.fit_transform(latent_representations)
10
11 # Visualisation
12 plt.figure(figsize=(10, 8))
13 plt.scatter(latent_2d[:, 0], latent_2d[:, 1], alpha=0.6, s=30)
14 plt.xlabel('t-SNE 1')
15 plt.ylabel('t-SNE 2')
16 plt.title('Visualisation t-SNE de l'espace latent')
17 plt.grid(True, alpha=0.3)
18 plt.savefig('tsne_visualization.png', dpi=150)
19 plt.show()

```

Listing 8 – Visualisation avec t-SNE

17.1.2 UMAP (Uniform Manifold Approximation and Projection)

UMAP est plus rapide que t-SNE et préserve mieux la structure globale.

```

1 import umap
2
3 # UMAP
4 reducer = umap.UMAP(n_components=2, random_state=42)
5 latent_2d_umap = reducer.fit_transform(latent_representations)
6
7 # Visualisation
8 plt.figure(figsize=(10, 8))
9 plt.scatter(latent_2d_umap[:, 0], latent_2d_umap[:, 1], alpha=0.6, s=30)
10 plt.xlabel('UMAP 1')
11 plt.ylabel('UMAP 2')
12 plt.title('Visualisation UMAP de l'espace latent')
13 plt.grid(True, alpha=0.3)
14 plt.savefig('umap_visualization.png', dpi=150)
15 plt.show()

```

Listing 9 – Visualisation avec UMAP

17.2 SHAP Values pour l'Interprétabilité

SHAP (SHapley Additive exPlanations) explique les prédictions en attribuant une importance à chaque feature.

```
1 import shap
2
3 # Créer un explainer
4 explainer = shap.TreeExplainer(iso_forest)
5
6 # Calculer les SHAP values
7 shap_values = explainer.shap_values(X_test)
8
9 # Visualisation
10 shap.summary_plot(shap_values, X_test, feature_names=feature_names)
11 plt.savefig('shap_summary.png', dpi=150, bbox_inches='tight')
12 plt.show()
13
14 # Feature importance
15 shap.summary_plot(shap_values, X_test, plot_type="bar",
16                  feature_names=feature_names)
17 plt.savefig('shap_importance.png', dpi=150, bbox_inches='tight')
18 plt.show()
```

Listing 10 – SHAP pour Isolation Forest

18 Déploiement en Production

18.1 Pipeline MLOps

18.1.1 Architecture de Déploiement

1. **Entraînement Offline** : Modèle entraîné sur données historiques
2. **Validation** : Tests sur ensemble de validation
3. **Versioning** : Sauvegarde du modèle avec MLflow/DVC
4. **Déploiement** : API REST avec FastAPI/Flask
5. **Monitoring** : Suivi des performances en temps réel
6. **Réentraînement** : Mise à jour périodique du modèle

18.1.2 Exemple d'API avec FastAPI

```
1 from fastapi import FastAPI, HTTPException
2 from pydantic import BaseModel
3 import numpy as np
4 import joblib
5
6 app = FastAPI(title="Anomaly Detection API")
7
8 # Charger le modèle et le scaler
9 model = joblib.load('autoencoder_model.pkl')
10 scaler = joblib.load('scaler.pkl')
11
12 class PredictionInput(BaseModel):
13     air_temp: float
14     process_temp: float
15     rotational_speed: float
16     torque: float
17     tool_wear: float
18
19 class PredictionOutput(BaseModel):
20     is_anomaly: bool
21     reconstruction_error: float
22     confidence: float
23
24 @app.post("/predict", response_model=PredictionOutput)
25 async def predict_anomaly(input_data: PredictionInput):
26     try:
27         # Préparer les données
28         features = np.array([
29             input_data.air_temp,
30             input_data.process_temp,
31             input_data.rotational_speed,
32             input_data.torque,
33             input_data.tool_wear
34         ])
35
36         # Normaliser
37         features_scaled = scaler.transform(features)
38
39         # Prédire
40         reconstruction = model.predict(features_scaled)
41         error = np.mean(np.square(features_scaled - reconstruction))
42
43         # Seuil ( à finir selon validation)
44         threshold = 0.05
45         is_anomaly = error > threshold
```

```

46         confidence = min(error / threshold, 2.0) if is_anomaly else 1.0 - (error
47                               / threshold)
48
49         return PredictionOutput(
50             is_anomaly=bool(is_anomaly),
51             reconstruction_error=float(error),
52             confidence=float(confidence)
53         )
54
55     except Exception as e:
56         raise HTTPException(status_code=500, detail=str(e))
57
58 @app.get("/health")
59 async def health_check():
60     return {"status": "healthy"}
61
62 if __name__ == "__main__":
63     import uvicorn
64     uvicorn.run(app, host="0.0.0.0", port=8000)

```

Listing 11 – API de Détection d'Anomalies avec FastAPI

18.2 Monitoring et Drift Detection

18.2.1 Data Drift

Détection de changements dans la distribution des données d'entrée.

Tests statistiques :

- **Kolmogorov-Smirnov** : Compare deux distributions
- **Chi-squared** : Pour variables catégorielles
- **Population Stability Index (PSI)** :

$$PSI = \sum_{i=1}^n (P_i - Q_i) \times \ln \left(\frac{P_i}{Q_i} \right) \quad (53)$$

Interprétation PSI :

- $PSI < 0.1$: Pas de changement significatif
- $0.1 \leq PSI < 0.25$: Changement modéré
- $PSI \geq 0.25$: Changement significatif (réentraînement recommandé)

19 Annexes

19.1 Ressources Complémentaires

19.1.1 Datasets Publics pour la Détection d'Anomalies

1. **AI4I 2020 Predictive Maintenance**
<https://archive.ics.uci.edu/dataset/601/ai4i+2020+predictive+maintenance+dataset>
2. **NASA Bearing Dataset**
 Données de vibrations pour prédiction de pannes de roulements
3. **SKAB (Skoltech Anomaly Benchmark)**
 Données de capteurs industriels avec anomalies annotées
4. **Credit Card Fraud Detection**
 Dataset Kaggle pour détection de fraudes

19.2 Bibliothèques Python Recommandées

Bibliothèque	Usage	Installation
scikit-learn	Méthodes classiques	<code>pip install scikit-learn</code>
TensorFlow/Keras	Deep Learning	<code>pip install tensorflow</code>
PyTorch	Deep Learning	<code>pip install torch</code>
PyOD	Détection d'anomalies	<code>pip install pyod</code>
SHAP	Interprétabilité	<code>pip install shap</code>
MLflow	MLOps	<code>pip install mlflow</code>
Streamlit	Dashboards	<code>pip install streamlit</code>

TABLE 7 – Bibliothèques Python pour la détection d'anomalies

19.3 Checklist de Projet

Exercice

Checklist pour un Projet de Détection d'Anomalies

Phase 1 : Exploration

- ☐ Comprendre le contexte métier
- ☐ Analyser la distribution des données
- ☐ Identifier les types d'anomalies attendues
- ☐ Définir les métriques de succès

Phase 2 : Prétraitement

- ☐ Nettoyer les données (valeurs manquantes, outliers)
- ☐ Normaliser/Standardiser les features
- ☐ Feature engineering si nécessaire
- ☐ Split train/validation/test

Phase 3 : Modélisation

- ☐ Tester plusieurs algorithmes
- ☐ Optimiser les hyperparamètres
- ☐ Valider sur ensemble de test
- ☐ Analyser les erreurs

Phase 4 : Déploiement

- ☐ Créer une API
- ☐ Mettre en place le monitoring
- ☐ Définir un processus de réentraînement
- ☐ Documenter le projet

20 Études de Cas Supplémentaires

20.1 Cas 1 : Détection de Fraudes Bancaires

20.1.1 Contexte

Les transactions frauduleuses représentent moins de 0.1% des transactions totales, créant un déséquilibre extrême.

Caractéristiques du problème :

- Déséquilibre extrême (99.9% normal, 0.1% fraude)
- Données sensibles (anonymisées via PCA)
- Besoin de temps de réponse rapide
- Coût élevé des faux négatifs

20.1.2 Approche Recommandée

1. Prétraitement :

- Normalisation des montants
- Feature engineering temporel (heure, jour de la semaine)
- Agrégation par client (historique)

2. Modélisation :

- Autoencodeur Dense pour apprendre les patterns normaux
- Isolation Forest pour rapidité
- Ensemble des deux méthodes

3. Évaluation :

- Privilégier le rappel (F2-score)
- Precision-Recall curve
- Coût métier des erreurs

20.2 Cas 2 : Maintenance Prédictive d'Éoliennes

20.2.1 Contexte

Prédire les pannes de composants d'éoliennes à partir de données de capteurs (vibrations, température, puissance).

Spécificités :

- Données de séries temporelles multivariées
- Conditions environnementales variables (vent, température)
- Plusieurs modes de défaillance
- Coût élevé des arrêts non planifiés

20.2.2 Solution Proposée

```
1 import pandas as pd
2 import numpy as np
3 from sklearn.preprocessing import StandardScaler
4 from tensorflow import keras
5 from tensorflow.keras import layers
6
7 # 1. Chargement des données
8 df = pd.read_csv('wind_turbine_data.csv')
```

```

9
10 # Features : vitesse vent, temp rature, vibrations, puissance
11 features = ['wind_speed', 'temperature', 'vibration_x',
12             'vibration_y', 'vibration_z', 'power_output']
13
14 # 2. Cr ation de s quences temporelles
15 def create_sequences(data, seq_length=100):
16     sequences = []
17     for i in range(len(data) - seq_length):
18         sequences.append(data[i:i+seq_length])
19     return np.array(sequences)
20
21 X = df[features].values
22 scaler = StandardScaler()
23 X_scaled = scaler.fit_transform(X)
24
25 # S quences de 100 timesteps
26 X_sequences = create_sequences(X_scaled, seq_length=100)
27
28 # 3. LSTM Autoencoder
29 n_timesteps = 100
30 n_features = len(features)
31 latent_dim = 10
32
33 # Encodeur
34 encoder_inputs = layers.Input(shape=(n_timesteps, n_features))
35 x = layers.LSTM(64, return_sequences=True)(encoder_inputs)
36 x = layers.LSTM(32, return_sequences=False)(x)
37 latent = layers.Dense(latent_dim, activation='relu')(x)
38
39 # D codeur
40 decoder_inputs = layers.Input(shape=(latent_dim,))
41 x = layers.RepeatVector(n_timesteps)(decoder_inputs)
42 x = layers.LSTM(32, return_sequences=True)(x)
43 x = layers.LSTM(64, return_sequences=True)(x)
44 decoder_outputs = layers.TimeDistributed(layers.Dense(n_features))(x)
45
46 # Mod le complet
47 encoder = keras.Model(encoder_inputs, latent)
48 decoder = keras.Model(decoder_inputs, decoder_outputs)
49
50 autoencoder_inputs = layers.Input(shape=(n_timesteps, n_features))
51 encoded = encoder(autoencoder_inputs)
52 decoded = decoder(encoded)
53 autoencoder = keras.Model(autoencoder_inputs, decoded)
54
55 autoencoder.compile(optimizer='adam', loss='mse')
56
57 # 4. Entra nement
58 history = autoencoder.fit(
59     X_sequences, X_sequences,
60     epochs=50,
61     batch_size=32,
62     validation_split=0.2
63 )
64
65 # 5. D tecton d'anomalies
66 reconstructions = autoencoder.predict(X_sequences)
67 mse = np.mean(np.square(X_sequences - reconstructions), axis=(1,2))
68
69 # Seuil adaptatif bas sur conditions
70 threshold_base = np.percentile(mse, 95)
71

```

```
72 # Ajustement selon vitesse du vent (conditions normales vs extrêmes)
73 wind_speeds = df['wind_speed'].values[100:] # Aligner avec séquences
74 threshold_adjusted = threshold_base * (1 + 0.1 * (wind_speeds / wind_speeds.mean
    ()))
75
76 anomalies = mse > threshold_adjusted
77
78 print(f"Anomalies détectées : {anomalies.sum()} / {len(anomalies)}")
```

Listing 12 – Pipeline pour Éoliennes

21 Optimisation et Hyperparamètres

21.1 Grid Search et Random Search

21.1.1 Grid Search

Exploration exhaustive d'une grille de paramètres.

```

1 from sklearn.model_selection import GridSearchCV
2 from sklearn.ensemble import IsolationForest
3 from sklearn.metrics import make_scorer, f1_score
4
5 # D finir la grille de param tres
6 param_grid = {
7     'n_estimators': [50, 100, 200],
8     'max_samples': [128, 256, 512],
9     'contamination': [0.01, 0.05, 0.1],
10    'max_features': [0.5, 0.75, 1.0]
11 }
12
13 # Scorer personnalis
14 def anomaly_f1_scorer(estimator, X, y):
15     y_pred = estimator.fit_predict(X)
16     y_pred_binary = (y_pred == -1).astype(int)
17     return f1_score(y, y_pred_binary)
18
19 scorer = make_scorer(anomaly_f1_scorer)
20
21 # Grid Search
22 grid_search = GridSearchCV(
23     IsolationForest(random_state=42),
24     param_grid,
25     scoring=scorer,
26     cv=3,
27     n_jobs=-1,
28     verbose=2
29 )
30
31 grid_search.fit(X_train, y_train)
32
33 print("Meilleurs param tres :")
34 print(grid_search.best_params_)
35 print(f"Meilleur score : {grid_search.best_score_:.4f}")

```

Listing 13 – Grid Search pour Isolation Forest

21.1.2 Bayesian Optimization

Optimisation plus efficace que Grid Search.

```

1 import optuna
2 from sklearn.ensemble import IsolationForest
3 from sklearn.metrics import f1_score
4
5 def objective(trial):
6     # Sugg rer des hyperparam tres
7     n_estimators = trial.suggest_int('n_estimators', 50, 200)
8     max_samples = trial.suggest_int('max_samples', 128, 512)
9     contamination = trial.suggest_float('contamination', 0.01, 0.2)
10    max_features = trial.suggest_float('max_features', 0.5, 1.0)
11
12    # Cr er et entra ner le mod le
13    model = IsolationForest(

```

```

14         n_estimators=n_estimators,
15         max_samples=max_samples,
16         contamination=contamination,
17         max_features=max_features,
18         random_state=42
19     )
20
21     y_pred = model.fit_predict(X_train)
22     y_pred_binary = (y_pred == -1).astype(int)
23
24     # Calculer le score
25     score = f1_score(y_train, y_pred_binary)
26
27     return score
28
29 # Créer l'étude
30 study = optuna.create_study(direction='maximize')
31 study.optimize(objective, n_trials=100)
32
33 print("Meilleurs paramètres :")
34 print(study.best_params)
35 print(f"Meilleur F1-score : {study.best_value:.4f}")
36
37 # Visualisation
38 from optuna.visualization import plot_optimization_history,
39     plot_param_importances
40
41 plot_optimization_history(study)
42 plot_param_importances(study)

```

Listing 14 – Bayesian Optimization avec Optuna

21.2 AutoML pour Détection d'Anomalies

21.2.1 TPOT (Tree-based Pipeline Optimization Tool)

```

1 from tpot import TPOTClassifier
2 from sklearn.model_selection import train_test_split
3
4 # Préparer les données
5 X_train, X_test, y_train, y_test = train_test_split(
6     X, y, test_size=0.2, random_state=42, stratify=y
7 )
8
9 # TPOT
10 tpot = TPOTClassifier(
11     generations=5,
12     population_size=20,
13     cv=5,
14     random_state=42,
15     verbosity=2,
16     n_jobs=-1
17 )
18
19 # Recherche du meilleur pipeline
20 tpot.fit(X_train, y_train)
21
22 # Évaluation
23 score = tpot.score(X_test, y_test)
24 print(f"Score sur test : {score:.4f}")
25
26 # Exporter le meilleur pipeline

```

```
27 | tpot.export('best_pipeline.py')
```

Listing 15 – AutoML avec TPOT

22 Détection d'Anomalies en Temps Réel

22.1 Architecture Streaming

22.1.1 Pipeline avec Apache Kafka

```

1 from kafka import KafkaConsumer, KafkaProducer
2 import json
3 import numpy as np
4 import joblib
5
6 # Charger le modèle
7 model = joblib.load('anomaly_detector.pkl')
8 scaler = joblib.load('scaler.pkl')
9
10 # Consumer Kafka
11 consumer = KafkaConsumer(
12     'sensor_data',
13     bootstrap_servers=['localhost:9092'],
14     value_deserializer=lambda m: json.loads(m.decode('utf-8'))
15 )
16
17 # Producer pour les alertes
18 producer = KafkaProducer(
19     bootstrap_servers=['localhost:9092'],
20     value_serializer=lambda v: json.dumps(v).encode('utf-8')
21 )
22
23 print("Démarrage de la détection en temps réel...")
24
25 for message in consumer:
26     try:
27         # Extraire les données
28         data = message.value
29         features = np.array([[
30             data['temperature'],
31             data['pressure'],
32             data['vibration'],
33             data['speed']
34         ]])
35
36         # Normaliser
37         features_scaled = scaler.transform(features)
38
39         # Prédire
40         prediction = model.predict(features_scaled)[0]
41
42         # Si anomalie, envoyer une alerte
43         if prediction == -1:
44             alert = {
45                 'timestamp': data['timestamp'],
46                 'sensor_id': data['sensor_id'],
47                 'anomaly_type': 'detected',
48                 'features': data
49             }
50             producer.send('anomaly_alerts', value=alert)
51             print(f"Anomalie détectée : {data['sensor_id']}")
52
53     except Exception as e:
54         print(f"Erreur : {e}")
55         continue

```

Listing 16 – Consumer Kafka pour Détection en Temps Réel

22.2 Détection Incrémentale

22.2.1 Online Learning

Pour s'adapter aux changements de distribution (concept drift).

```
1 from river import anomaly
2 from river import preprocessing
3
4 # Mod le incr mental (Half-Space Trees)
5 model = anomaly.HalfSpaceTrees(
6     n_trees=10,
7     height=8,
8     window_size=250,
9     seed=42
10 )
11
12 # Pr ocessing
13 scaler = preprocessing.StandardScaler()
14
15 # Simulation de stream
16 for i, (x, y) in enumerate(stream_data):
17     # Normaliser
18     x_scaled = scaler.learn_one(x).transform_one(x)
19
20     # Pr dire
21     score = model.score_one(x_scaled)
22     is_anomaly = score > threshold
23
24     # Apprendre (uniquement si normal)
25     if not is_anomaly:
26         model.learn_one(x_scaled)
27
28     # Afficher les anomalies
29     if is_anomaly:
30         print(f"Anomalie      t={i}: score={score:.4f}")
```

Listing 17 – Apprentissage Incrémental

23 Conclusion et Perspectives

23.1 Récapitulatif

Ce cours a présenté de manière exhaustive les algorithmes de détection d'anomalies pour la maintenance prédictive :

Autoencodeurs :

- Autoencodeur Dense : Excellent pour données tabulaires haute dimension
- Autoencodeur LSTM : Idéal pour séries temporelles et patterns séquentiels

Méthodes Classiques :

- Isolation Forest : Rapide, scalable, bon pour anomalies globales
- One-Class SVM : Frontière flexible, théorie solide
- LOF : Détection d'anomalies locales, basé sur la densité

23.2 Tendances Futures

23.2.1 Deep Learning Avancé

1. **Transformers pour Séries Temporelles**

Attention mechanisms pour capturer les dépendances à très long terme

2. **Variational Autoencoders (VAE)**

Modélisation probabiliste de l'espace latent

3. **Generative Adversarial Networks (GAN)**

Génération de données normales pour augmentation

4. **Graph Neural Networks**

Pour données structurées en graphes (réseaux de capteurs)

23.2.2 Explainability (XAI)

Importance croissante de l'interprétabilité :

- SHAP, LIME pour expliquer les prédictions
- Attention maps pour visualiser ce que le modèle "regarde"
- Counterfactual explanations

23.2.3 AutoML et Neural Architecture Search

Automatisation de la conception de modèles :

- Recherche automatique d'architectures optimales
- Meta-learning pour transfert entre domaines
- Few-shot learning pour données limitées

23.3 Recommandations Finales

Astuce

Conseils pour un Projet Réussi :

1. **Comprendre le métier** : Collaborer avec les experts domaine
2. **Commencer simple** : Baseline avec méthodes classiques
3. **Itérer** : Améliorer progressivement
4. **Valider rigoureusement** : Tests sur données réelles
5. **Monitorer en production** : Détecter le drift
6. **Documenter** : Code, décisions, résultats
7. **Communiquer** : Expliquer les résultats aux stakeholders

23.4 Ressources pour Aller Plus Loin

Livres :

- *Deep Learning* - Goodfellow, Bengio, Courville
- *Hands-On Machine Learning* - Aurélien Géron
- *Outlier Analysis* - Charu Aggarwal

Cours en ligne :

- Coursera : Deep Learning Specialization (Andrew Ng)
- Fast.ai : Practical Deep Learning
- Kaggle : Anomaly Detection Courses

Conférences :

- NeurIPS, ICML, ICLR (Machine Learning)
- KDD (Knowledge Discovery and Data Mining)
- AAAI (Artificial Intelligence)

Important

Message Final :

La détection d'anomalies est un domaine en constante évolution. Les techniques présentées dans ce cours constituent une base solide, mais il est essentiel de rester à jour avec les dernières recherches et de toujours adapter les méthodes au contexte spécifique de votre application.

Bonne chance dans vos projets de maintenance prédictive !

Références

- [1] Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
- [2] Liu, F. T., Ting, K. M., & Zhou, Z. H. (2008). Isolation forest. *IEEE ICDM*, 413-422.
- [3] Schölkopf, B., et al. (2001). Estimating the support of a high-dimensional distribution. *Neural computation*, 13(7), 1443-1471.
- [4] Breunig, M. M., et al. (2000). LOF : identifying density-based local outliers. *ACM sigmod record*, 29(2), 93-104.
- [5] Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural computation*, 9(8), 1735-1780.
- [6] Chandola, V., Banerjee, A., & Kumar, V. (2009). Anomaly detection : A survey. *ACM computing surveys*, 41(3), 1-58.